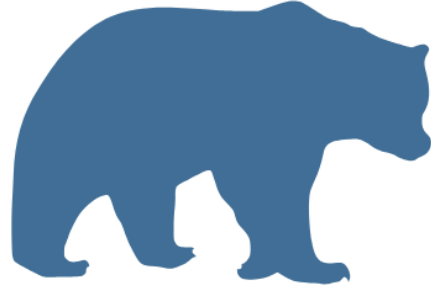


Introducción a la librería Pandas



¿Qué es pandas?

Pandas es una librería de Python de código abierto especializada en el manejo y análisis de estructuras de datos. Es rápida, potente, flexible y fácil de usar.

Tiene el objetivo de convertirse en la herramienta de análisis/manipulación de datos de código abierto más potente y flexible.

Características

- Define **nuevas estructuras de datos** basadas en los arrays de NumPy pero con nuevas funciones: Series, DataFrames y Panel.
- Permite **leer y escribir** fácilmente **ficheros** en formato CSV, Excel y bases de datos SQL.
- Permite **acceder** a los datos **mediante índices o nombres** para filas y columnas.
- Ofrece métodos para **reordenar, dividir y combinar** conjuntos de datos.
- Permite trabajar con **series temporales** (indexadas por un timestamp/marca temporal).
- Realiza todas estas operaciones de manera muy **eficiente**.

Estructuras de datos de pandas

Convenio de importación de Pandas (recordamos también el de Numpy)

```
import pandas as pd
import numpy as np
```

Pandas dispone de dos estructuras de datos principales:

- **Series:** Estructura de una dimensión.
- **DataFrame:** Estructura de dos dimensiones (tablas).

Estas estructuras se construyen a partir de arrays de la librería NumPy, añadiendo nuevas funcionalidades.

Series

Una serie es un objeto unidimensional de estilo array que contiene una secuencia de valores del mismo tipo y un array asociado de etiquetas de datos, que corresponde a su índice. Las series son homogéneas, es decir, sus elementos tienen que ser del mismo tipo, y su tamaño es inmutable, no se puede cambiar, aunque sí su contenido.

Así sería la creación de una serie con índice, que identifique cada punto de datos con una etiqueta:

```
obj = pd.Series([4, 7, -5, 3], index=["d","b","a","c"])
```

Ahora, podemos acceder a los elementos de la serie usando los índices:

```
print(obj["a"]) #Mostrará el valor -5

obj["d"] = 6
print(obj) #Mostrará lo que pongo a continuación
```

d	6
b	7
a	-5
c	3
dtype: int64	

En este caso, ["d","b","a","c"] se interpreta como lista de **índices** aunque contenga cadena de texto.

El objeto Series más sencillo que se puede crear es uno formado sólo por un array de datos.

```
obj = pd.Series([4, 7, -5, 3])
```

Si el índice no se especifica como en este caso, se crea uno predeterminado formado por los enteros 0 a N-1 (siendo N el número de elementos de la serie).

Métodos para ver las características de una serie:

- **s.size**: Devuelve el número total de elementos de la serie s.
- **s.index**: Devuelve un objeto index que representa las etiquetas de los elementos de s.
- **s.dtype** : Devuelve el tipo de datos de los elementos de la serie s.

Otra forma de ver las series podría ser la de un **diccionario ordenado de longitud fija**, ya que se asignan índices a datos.

Crear serie a partir de diccionario
<pre>dic = {"Ohio": 35000, "Texas": 71000, "Oregon": 16000, "Utah": 5000} new_series = pd.Series(dic)</pre>
Crear diccionario a partir de serie
<pre>new_dic = new_series.to_dict()</pre>

Si queremos crear una serie a partir de un diccionario pero modificando las claves, se puede hacer de la siguiente forma

Crear serie a partir de diccionario cambiando índices
<pre>#Tenemos este diccionario dic = {"Ohio": 35000, "Texas": 71000, "Oregon": 16000, "Utah": 5000} #Queremos que la serie tenga este nuevo índice estados = ["California", "Ohio", "Oregon", "Texas"] new_series_modif = pd.Series(dic, index=estados)</pre>

```
print(new_serie_modif)
```

```
California    NaN
Ohio          35000.0
Oregon        16000.0
Texas         71000.0
dtype: float64
```

En esta serie resultante tendremos “California” con un valor **NaN** asociado (Not a Number), que es la expresión que **pandas** usa para indicar de un **valor ausente**. Por otro lado, “Utah”, que sí estaba en el diccionario, no aparecerá en la nueva serie porque en el índice no estaba ese valor, así que **se excluye**.

Para saber si en una serie hay valores ausentes/nulos/faltantes se pueden emplear las funciones **isna** y **notna**

```
print(pd.isna(new_serie_modif))
```

```
California    True
Ohio          False
Oregon        False
Texas         False
dtype: bool
```

```
print(pd.notna(new_serie_modif))
```

```
California    False
Ohio          True
Oregon        True
Texas         True
dtype: bool
```

Por último, se le puede poner un nombre, tanto al objeto Series como al índice, con el atributo **name**

```
dic = {"Pedro": 183, "Ana": 171, "Raquel": 157, "Nico": 165}
```

```
new_serie = pd.Series(dic)
```

```

new_serie.name = "Altura_cm"
new_serie.index.name = "Nombres"

print(new_serie)

```

Nombres	
Pedro	183
Ana	171
Raquel	157
Nico	165

```

Name: Altura_cm, dtype: int64

```

DataFrame

Un **dataframe** representa una tabla rectangular de datos y contiene una colección de columnas ordenada y con nombre. Cada columna puede tener un tipo de valor distinto (numérico, cadena de texto, booleano...). El objeto dataframe tiene un índice para las filas y otro para las columnas. Puede verse como un conjunto de objetos Series que comparten el mismo índice.

```

import pandas as pd

datos = {"ciudad":["Almería", "Zafra", "Madrid", "Salamanca"],
        "temp_max":[38, 40, 36, 32],
        "temp_min":[12, -2, -8, -5]}

frame = pd.DataFrame(datos)

frame.index.name = "Indice"
frame.columns.name = "Características"

print(frame)

```

	Características	ciudad	temp_max	temp_min
Indice				
0		Almería	38	12
1		Zafra	40	-2
2		Madrid	36	-8
3		Salamanca	32	-5

Como vemos en el ejemplo anterior, el índice de las filas se asigna de manera automática (tal y como pasa con las series si no se especifica). Las columnas se ordenan según están en la declaración de datos, a no ser que se especifique una secuencia concreta como en el ejemplo siguiente:

```
import pandas as pd
```

```
datos = {"ciudad": ["Almería", "Zafra", "Madrid", "Salamanca"],  
        "temp_max": [38, 40, 36, 32],  
        "temp_min": [12, -2, -8, -5]}
```

```
frame = pd.DataFrame(datos, columns=["temp_min", "temp_max",  
"ciudad"])
```

```
print(frame)
```

	temp_min	temp_max	ciudad
0	12	38	Almería
1	-2	40	Zafra
2	-8	36	Madrid
3	-5	32	Salamanca

#Si se indica una columna que no está en el diccionario, se rellena con valores faltantes

```
frame2 = pd.DataFrame(datos, columns=["temp_min", "humedad",  
"ciudad"])
```

```
print(frame2)
```

	temp_min	humedad	ciudad
0	12	NaN	Almería
1	-2	NaN	Zafra
2	-8	NaN	Madrid
3	-5	NaN	Salamanca

A diferencia de las series, los dataframes no tienen atributo name (aunque si podemos ponerle nombre al índice de las filas y columnas como hemos visto en el ejemplo previo).

Algunos métodos útiles para dataframes:

```
frame.head() #Devuelve las primeras cinco filas  
frame.tail() #Devuelve las últimas cinco filas
```

```

frame["nombre_columna"] #Devuelve la columna "nombre_columna" como una serie
frame.nombre_columna    #Devuelve la columna "nombre_columna" como una serie
                        (esta segunda forma de consultar columna no funciona
                        siempre, depende del nombre de la columna)
frame.loc["nombre_fila"] #Devuelve la fila con indice = "nombre_fila"
frame.iloc[x]            #Devuelve la fila de la posición x

del frame["nombre_columna"] #Elimina columna "nombre_columna"

frame.T                  #Traspone el dataframe (intercambia filas por columnas)

frame.to_numpy           #Convierte el DataFrame en un numpy array.

```

Podemos modificar el contenido de las columnas dándole el mismo valor a todas las filas por medio de la asignación, o variando el contenido empleando arrays.

```

import pandas as pd
import numpy as np

datos = {"nombre":["Jose", "Juan", "Carlos", "Salvador"],
         "estado_civil":["casado", "viudo", "soltero", "casado"],
         "edad":[32, 83, 49, 64]}

frame = pd.DataFrame(datos, columns=["nombre", "estado_civil",
                                     "hijos"])

print(frame)

           nombre estado_civil hijos
0         Jose      casado    NaN
1         Juan      viudo    NaN
2        Carlos    soltero    NaN
3    Salvador      casado    NaN

#Asignamos mismo valor por asignación
frame["hijos"] = 1

print(frame)

```

	nombre	estado_civil	hijos
0	Jose	casado	1
1	Juan	viudo	1
2	Carlos	soltero	1
3	Salvador	casado	1

```
#Asignamos array de valores con longitud = num.filas del dataframe
frame["hijos"] = np.arange(4.)
```

```
print(frame)
```

	nombre	estado_civil	hijos
0	Jose	casado	0.0
1	Juan	viudo	1.0
2	Carlos	soltero	2.0
3	Salvador	casado	3.0

```
#Asignamos serie, solo se asigna para los indices existentes
val = pd.Series([1,2,4,3], index=["primero", 3, "segundo", "tercero"])
```

```
frame["hijos"] = val
print(frame)
```

	nombre	estado_civil	hijos
0	Jose	casado	NaN
1	Juan	viudo	NaN
2	Carlos	soltero	NaN
3	Salvador	casado	2.0

```
#Asignar columna que no existe crea una nueva
#OJO!! no se pueden crear con frame.nueva_columna
```

```
frame["mascotas"] = frame["hijos"]
print(frame)
```

	nombre	estado_civil	hijos	mascotas
0	Jose	casado	NaN	NaN
1	Juan	viudo	NaN	NaN
2	Carlos	soltero	NaN	NaN
3	Salvador	casado	2.0	2.0

Para terminar con los dataframes, vamos a comentar la posibilidad de crearlos a partir de diccionarios anidado de diccionarios


```
import pandas as pd
import numpy as np

poblaciones = {"Ohio": {2000:1.5, 2001:1.7, 2002:3.6},
               "Nevada": {2001:1.4, 2002:2.9}}

frame = pd.DataFrame(poblaciones)

print(frame)
```

```
      Ohio  Nevada
2000    1.5     NaN
2001    1.7     1.4
2002    3.6     2.9
```

TABLA RESUMEN: Posibles entradas de datos para constructor DataFrame

Tipo de estructura de datos	Conversión a DataFrame	Notas
ndarray de dos dimensiones	Se convierte directamente en un DataFrame	Puede incluir etiquetas opcionales para filas y columnas
Diccionario de arrays, listas o tuplas	Cada secuencia se convierte en una columna	Todas deben tener la misma longitud
Diccionario de Series	Cada Series se convierte en una columna	Los índices de las Series se combinan para formar el índice de las filas final (a no ser que se pase un índice explícito)
Diccionario de diccionarios	Cada diccionario interno se convierte en una columna	Las claves internas se convierten en índices de filas

Objetos índice

Los objetos índice de pandas albergan etiquetas de ejes (incluyendo el nombre de las columnas de un dataframe) y otros metadatos (como los nombres de los ejes). Cualquier array u otra secuencia de etiquetas que se utilice al construir una serie o dataframe se convierte internamente en un índice. Estos objetos son **inmutables**, por lo que no pueden ser modificados por el usuario.

```
import pandas as pd
import numpy as np
```

```
obj = pd.Series(np.arange(3), index=["a", "b", "c"])
index = obj.index

index[1]="d"

index[1]="d"
~~~~~^^^
File ~~~~~
raise TypeError("Index does not support mutable o
TypeError: Index does not support mutable operations
```

Un índice de pandas se comporta como un conjunto de tamaño fijo, pero que además **puede contener etiquetas duplicadas**. Si hacemos selecciones de la etiqueta que aparece varias veces, se incluirán en la selección todas las apariciones de dicha etiqueta.

Métodos y propiedades del objeto índice:

Método / propiedad	Descripción
append()	Concatena con objetos Index adicionales, produciendo un nuevo índice.
difference()	Calcula la diferencia de conjuntos como un Index. Retorna los elementos que están en el índice original pero no en el índice pasado como argumento.
intersection()	Calcula la intersección entre conjuntos de índices y devuelve los elementos comunes.
union()	Retorna la unión de dos Index, combinando los elementos sin duplicados.
isin()	Devuelve un array booleano indicando si cada elemento del Index está presente en una lista u otro Index.
delete()	Devuelve un nuevo Index con un elemento eliminado en la posición especificada.
drop()	Devuelve un nuevo Index con ciertos valores eliminados. Funciona con valores en lugar de posiciones.
insert()	Inserta un nuevo valor en una posición específica dentro del Index, devolviendo un nuevo Index.
is_monotonic	Propiedad booleana que indica si los valores del Index están ordenados de forma creciente o decreciente.

is_unique	Propiedad booleana que indica si todos los valores en el Index son únicos (sin duplicados).
unique()	Retorna un array con los valores únicos en el Index.